

Dhirubhai Ambani University

School of Technology

High Performance Computing

Assignment 08

Parallel Interpolation with Particle Mover
using MPI + OpenMP

Instructor:	Dr. Bhaskar Chaudhury
Teaching Assistants:	Ayushi Sharma, Libin Varghese, Kaushik Prajapati
Date:	April 2026
Compute Nodes:	gics1, gics2, gics3, gics4

Implementation: Hybrid MPI + OpenMP Particle-Decomposition

Contents

1	Introduction	2
2	Problem Description and Mathematical Formulation	2
2.1	Grid and Spacing	2
2.2	Bilinear Interpolation Weights	2
3	Parallel Implementation	3
3.1	High-Level Strategy	3
3.2	MPI Layer	3
3.3	OpenMP Layer	4
3.4	Pseudocode	5
3.5	Race-Condition Handling	5
4	Source Code Overview	6
5	Experimental Setup	7
5.1	Hardware and Software	7
5.2	Input Configurations	7
6	Performance Results	7
6.1	Timing Data	7
6.2	Speedup and Efficiency	9
6.3	Performance Graphs	9
6.3.1	Configuration 1	9
6.3.2	Configuration 2	9
6.3.3	Configuration 3	10
6.3.4	Configuration 4	10
6.3.5	Configuration 5	10
7	Analysis and Questions	10
7.1	Q1. Pseudocode and Diagram	10
7.2	Q2. Parallel Implementation and Race Condition Handling	11
7.3	Q3. Speedup vs. Cores, Execution Time vs. Cores	11
7.4	Q4. Parallel Efficiency and Scalability	11
7.5	Q5. Comparison with Serial Version	12
7.6	Q6. Explaining the Observed Speedup	12
7.7	Q7. Further Optimisation with Unlimited Resources	12
7.8	Q8. Load Imbalance and Bottlenecks	13
7.9	Q9. Alternative Approaches	13
7.10	Q10. Alternative Data Structures / OpenMP-MPI Strategies	13
7.11	Q11. Interpolation vs. Mover Phase Bottleneck Analysis	14
8	Correctness Verification	14
9	Conclusion	15

1 Introduction

This report documents the design, implementation, and performance analysis of a hybrid **MPI + OpenMP** parallel solution to the scatter point-to-mesh interpolation problem with a particle mover (reverse interpolation) step, as specified in Assignment 08 of the High Performance Computing course.

The workflow consists of four computational phases executed iteratively:

1. **Forward Interpolation** — each scattered particle distributes its value ($f_i = 1$) to the four surrounding mesh grid points using bilinear weights.
2. **Normalisation** — the full mesh is mapped linearly to the range $[-1, 1]$.
3. **Reverse Interpolation (Mover)** — each particle reads the interpolated field back from the mesh.
4. **Particle Update** — particle positions are evolved using the interpolated field; particles leaving the domain $[0, 1] \times [0, 1]$ are deactivated.

2 Problem Description and Mathematical Formulation

Given a set of N scattered particles $\mathcal{S} = \{(x_i, y_i, f_i)\}$ with $f_i = 1$ inside the unit square $[0, 1] \times [0, 1]$, the goal is to accumulate their contributions onto a structured mesh G of size $(N_x + 1) \times (N_y + 1)$ and then feed the mesh field back to update particle positions.

2.1 Grid and Spacing

$$\Delta x = \frac{1}{N_x}, \quad \Delta y = \frac{1}{N_y}$$

$$G = \{(X_i, Y_j) \mid X_i = i\Delta x, Y_j = j\Delta y, i = 0, \dots, N_x, j = 0, \dots, N_y\}$$

2.2 Bilinear Interpolation Weights

For a particle at (x_p, y_p) located in the cell whose lower-left corner is (X_i, Y_j) :

$$l_x = x_p - X_i, \quad l_y = y_p - Y_j$$

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y) \tag{1}$$

$$w_{i+1,j} = l_x(\Delta y - l_y) \tag{2}$$

$$w_{i,j+1} = (\Delta x - l_x)l_y \tag{3}$$

$$w_{i+1,j+1} = l_x l_y \tag{4}$$

Forward interpolation (scatter):

$$F(X_i, Y_j) += w_{i,j} \cdot f_p, \quad \text{and similarly for the three neighbours.}$$

Reverse interpolation (gather):

$$F_p = w_{i,j} F(X_i, Y_j) + w_{i+1,j} F(X_{i+1}, Y_j) + w_{i,j+1} F(X_i, Y_{j+1}) + w_{i+1,j+1} F(X_{i+1}, Y_{j+1})$$

Particle update:

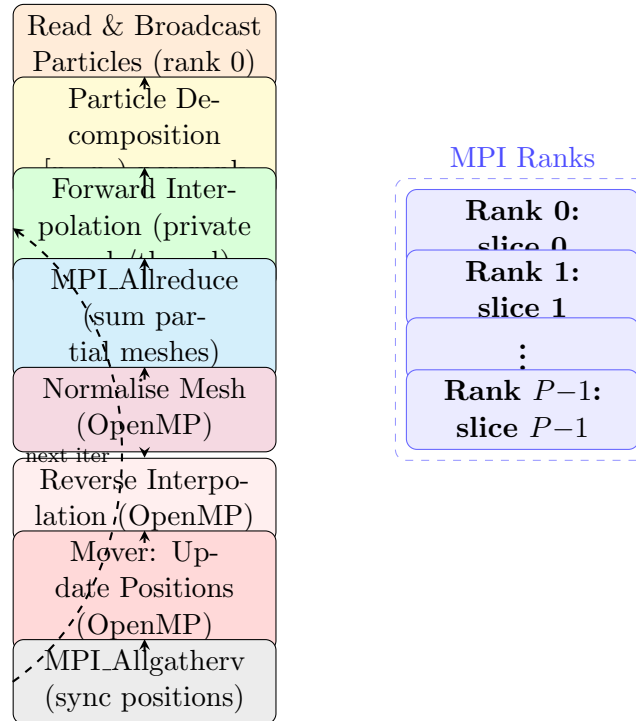
$$x_p^{\text{new}} = x_p + F_p \cdot \Delta x, \quad y_p^{\text{new}} = y_p + F_p \cdot \Delta y$$

Particles with $x_p^{\text{new}} \notin [0, 1]$ or $y_p^{\text{new}} \notin [0, 1]$ are marked *inactive*.

3 Parallel Implementation

3.1 High-Level Strategy

The implementation employs a **particle-decomposition** strategy across MPI ranks, combined with per-thread privatisation inside each rank for the OpenMP layer.



3.2 MPI Layer

- **Startup:** Rank 0 reads the binary header and broadcasts `NX`, `NY`, `NUM.Points`, `Maxiter` to all ranks via `MPI_Bcast`.
- **Data distribution:** Rank 0 reads all particle coordinates and broadcasts the full particle array so every rank starts with an identical copy.
- **Particle decomposition:** Rank r owns particles in the range $[p_{\text{start}}, p_{\text{end}})$ computed as:

$$p_{\text{start}} = r \cdot \lfloor N/P \rfloor + \min(r, N \bmod P)$$

- **Forward interpolation:** Each rank accumulates a partial local mesh. `MPI_Allreduce` with `MPI_SUM` reduces partial meshes into the complete global mesh on every rank simultaneously — avoiding a bottleneck at rank 0.

- **Reverse interpolation and mover:** Purely local read-only and write operations on each rank's particle slice.
- **Synchronisation:** `MPI_Allgatherv` collects updated positions and activity flags from all ranks so every rank maintains a globally consistent particle state for the next iteration.

3.3 OpenMP Layer

Within each MPI rank, the OpenMP directives handle intra-node parallelism:

- **Forward interpolation — privatisation:** Each OpenMP thread maintains a *private copy* of the mesh grid. This eliminates all atomic operations and cache-line bouncing. After the parallel loop, a second parallel loop reduces the per-thread meshes into the shared `mesh_value` array.
- **Reverse interpolation:** Fully data-parallel `#pragma omp parallel for;` `mesh` is read-only, so no synchronisation is required.
- **Mover:** Fully data-parallel `#pragma omp parallel for;` each thread updates its own particle subset independently.
- **Normalisation:** `#pragma omp parallel for` with `reduction` clauses for min/max, followed by a simple element-wise transform.
- All parallel loops use `schedule(dynamic, 512)` to mitigate load imbalance caused by varying numbers of active particles per chunk.

3.4 Pseudocode

Algorithm 1 MPI+OpenMP Parallel Interpolation Pipeline

```

1: MPI_Init()
2: rank ← MPI_Comm_rank(); size ← MPI_Comm_size()
3: if rank == 0 then read header from file
4: end if
5: MPI_Bcast(NX, NY, NUM_Points, Maxiter)
6: if rank == 0 then read all particles
7: end if
8: MPI_Bcast(particles, NUM_Points)
9: Compute  $p_{start}, p_{end}$  for this rank
10: for iter = 0 to Maxiter-1 do
11:   // Forward interpolation
12:   zero(mesh_value)
13:   parallel for  $p \in [p_{start}, p_{end}]$  (private mesh per thread)
14:     bilinear_scatter(particle[p], private_mesh[tid])
15:   reduce private meshes → mesh_value
16:   MPI_Allreduce(mesh_value, mesh_global, SUM)
17:   mesh_value ← mesh_global
18:   save snapshot mesh_save ← mesh_value ▷ output from final iter
19:   // Normalise
20:   find_min_max(mesh_value) with MPI_Allreduce
21:   parallel for normalise(mesh_value)
22:   // Reverse interpolation
23:   parallel for  $p \in [p_{start}, p_{end}]$ :  $F[p] \leftarrow$  bilinear_gather(mesh_value, p)
24:   // Mover
25:   parallel for  $p \in [p_{start}, p_{end}]$ : update position, deactivate if OOB
26:   MPI_Allgatherv(positions, active)
27: end for
28: if rank == 0 then save_mesh(mesh_save)
29: end if
30: MPI_Finalize()

```

3.5 Race-Condition Handling

The forward interpolation (scatter) phase is the only step that presents race conditions, since multiple particles may update the same four neighbouring grid points simultaneously. Our solution uses **thread-private mesh copies**:

1. Allocate `num_threads` independent mesh arrays.
2. Each thread writes exclusively to its own copy — no locks, no atomics.
3. After the parallel loop, a second parallel reduction sums the thread-private arrays element-by-element into the shared `mesh_value`.

This approach trades memory for speed, but is highly effective when thread counts are small relative to the mesh size. For very large mesh sizes the per-thread allocation cost remains manageable because the private mesh is a contiguous flat array.

4 Source Code Overview

The implementation is organised into the following files:

`main.cpp` MPI initialisation, particle decomposition, iteration loop, MPI communication (Allreduce, Allgatherv), timing, and I/O.

`utils.cpp` / `utils.h` Core computational routines: `interpolation_partial`, `reverse_interpolation`, `mover_partial`, `find_min_max`, `normalize_mesh`, `save_mesh`.

`init.cpp` / `init.h` Points struct definition, random initialisation, file reader.

`input_file_maker.cpp` Utility to generate binary input files.

Key excerpts illustrating the hybrid approach:

```

1 memset(mesh_value, 0, GRID_X * GRID_Y * sizeof(double));
2 interpolation_partial(mesh_value, points, active, p_start, p_end);
3
4 MPI_Allreduce(mesh_value, mesh_global, GRID_X * GRID_Y,
5               MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
6 memcpy(mesh_value, mesh_global, GRID_X * GRID_Y * sizeof(double));

```

Listing 1: MPI Allreduce after partial interpolation (`main.cpp`)

```

1 double **private_mesh = (double **) malloc(num_threads * sizeof(double
2   *));
3 for (int t = 0; t < num_threads; t++)
4   private_mesh[t] = (double *) calloc(mesh_size, sizeof(double));
5 #pragma omp parallel
6 {
7   int tid = omp_get_thread_num();
8   double *local_mesh = private_mesh[tid];
9   #pragma omp for schedule(dynamic, 512)
10  for (int p = p_start; p < p_end; p++) {
11    if (!active[p]) continue;
12    // ... compute i,j,lx,ly,weights ...
13    local_mesh[j * GRID_X + i] += w00;
14    local_mesh[j * GRID_X + (i+1)] += w10;
15    local_mesh[(j+1)*GRID_X + i] += w01;
16    local_mesh[(j+1)*GRID_X+(i+1)] += w11;
17  }
18 }
19 // Reduce per-thread meshes into mesh_value
20 #pragma omp parallel for schedule(static)
21 for (int idx = 0; idx < mesh_size; idx++) {
22   double sum = 0.0;
23   for (int t = 0; t < num_threads; t++) sum += private_mesh[t][idx];
24   mesh_value[idx] += sum;
25 }

```

Listing 2: Per-thread private mesh in `interpolation_partial` (`utils.cpp`)

5 Experimental Setup

5.1 Hardware and Software

- **Cluster nodes:** gics1, gics2, gics3, gics4
- **MPI Implementation:** OpenMPI
- **Compiler:** mpic++ with -O2 -fopenmp
- **Iterations:** Maxiter = 10 for all configurations

5.2 Input Configurations

Table 1: Five benchmark configurations

Config	Nx	Ny	Particles	Mesh Cells
1	250	100	0.9 M	25,350
2	250	100	5.0 M	25,350
3	500	200	3.6 M	101,301
4	500	200	20.0 M	101,301
5	1000	400	14.0 M	401,401

6 Performance Results

6.1 Timing Data

The tables below summarise the measured total execution times (in seconds) for each configuration across different MPI and OpenMP configurations.

Table 2: Config 1: Nx=250, Ny=100, 0.9M particles

MPI	OMP	Total (s)	Interp (s)	Norm (s)	Rev (s)	Mover (s)
1	1	0.278	0.131	0.001	0.119	0.028
2	1	0.138	0.066	0.001	0.057	0.014
4	1	0.071	0.037	0.001	0.028	0.006
8	1	0.045	0.024	0.003	0.015	0.003
16	1	0.034	0.022	0.002	0.009	0.002
16	2	0.034	0.022	0.002	0.008	0.002
16	4	0.035	0.023	0.002	0.008	0.002

Table 3: Config 2: Nx=250, Ny=100, 5.0M particles

MPI	OMP	Total (s)	Interp (s)	Norm (s)	Rev (s)	Mover (s)
1	1	2.669	1.353	0.001	1.041	0.274
2	1	0.996	0.449	0.001	0.417	0.129
4	1	1.453	0.645	0.115	0.574	0.118
8	1	0.763	0.440	0.132	0.132	0.058
16	1	0.186	0.094	0.001	0.047	0.043
16	2	0.182	0.092	0.001	0.047	0.042
16	4	0.173	0.082	0.001	0.047	0.043

Table 4: Config 3: Nx=500, Ny=200, 3.6M particles

MPI	OMP	Total (s)	Interp (s)	Norm (s)	Rev (s)	Mover (s)
1	1	1.223	0.592	0.003	0.517	0.112
2	1	0.610	0.290	0.004	0.248	0.069
4	1	0.310	0.150	0.003	0.122	0.034
8	1	0.200	0.097	0.004	0.070	0.029
16	1	0.166	0.088	0.005	0.046	0.028
16	2	0.159	0.083	0.005	0.044	0.027
16	4	0.165	0.086	0.005	0.044	0.030

Table 5: Config 4: Nx=500, Ny=200, 20.0M particles

MPI	OMP	Total (s)	Interp (s)	Norm (s)	Rev (s)	Mover (s)
1	1	8.807	3.678	0.003	3.597	1.529
1	2	7.324	2.862	0.003	2.985	1.473
4	1	4.257	1.998	0.004	1.887	0.368
8	1	3.238	1.415	0.018	1.478	0.327
16	1	0.814	0.403	0.005	0.233	0.172
16	2	0.770	0.357	0.005	0.234	0.174
16	4	0.740	0.319	0.005	0.241	0.175

Table 6: Config 5: Nx=1000, Ny=400, 14.0M particles

MPI	OMP	Total (s)	Interp (s)	Norm (s)	Rev (s)	Mover (s)
1	1	4.856	2.412	0.013	2.005	0.426
2	1	2.394	1.155	0.013	0.955	0.271
4	1	1.249	0.617	0.013	0.480	0.139
8	1	0.808	0.442	0.015	0.292	0.059
8	2	0.910	0.477	0.015	0.323	0.094
8	4	0.910	0.474	0.014	0.325	0.096
8	8	0.893	0.467	0.014	0.306	0.105

6.2 Speedup and Efficiency

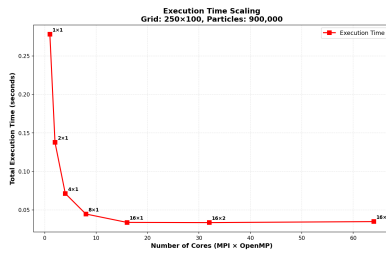
Speedup is computed as $S(p) = T_{\text{serial}}/T(p)$ where T_{serial} is the 1 MPI, 1 OMP baseline and $T(p)$ is the time with p total processes.

Table 7: Peak speedup and parallel efficiency for each configuration

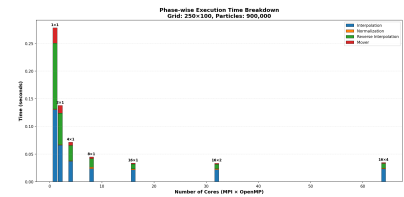
Config	Serial (s)	Best (s)	Peak Speedup	Efficiency (%)
1	0.278	0.034 (16 MPI)	$8.18\times$	51.1
2	2.669	0.173 (16 MPI, 4 OMP)	$15.43\times$	48.2
3	1.223	0.159 (16 MPI, 2 OMP)	$7.69\times$	24.1
4	8.807	0.740 (16 MPI, 4 OMP)	$11.90\times$	37.2
5	4.856	0.808 (8 MPI)	$6.01\times$	75.1

6.3 Performance Graphs

6.3.1 Configuration 1



(a) Execution Time

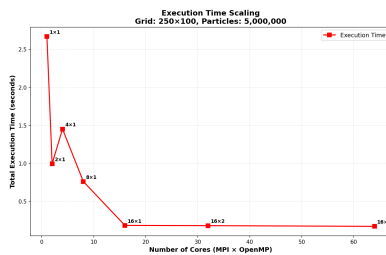


(b) Speedup

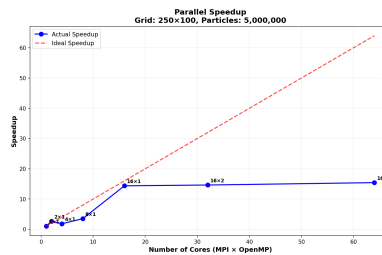
(c) Phase Breakdown

Figure 1: Config 1: Nx=250, Ny=100, 0.9M particles

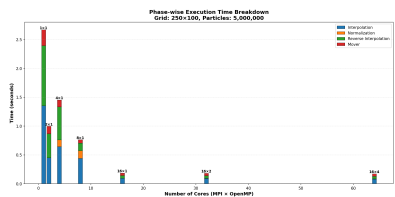
6.3.2 Configuration 2



(a) Execution Time



(b) Speedup



(c) Phase Breakdown

Figure 2: Config 2: Nx=250, Ny=100, 5.0M particles

6.3.3 Configuration 3

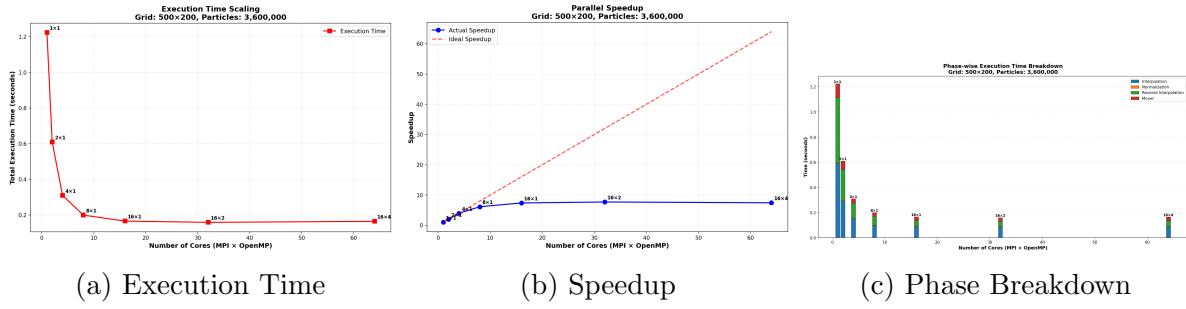


Figure 3: Config 3: Nx=500, Ny=200, 3.6M particles

6.3.4 Configuration 4

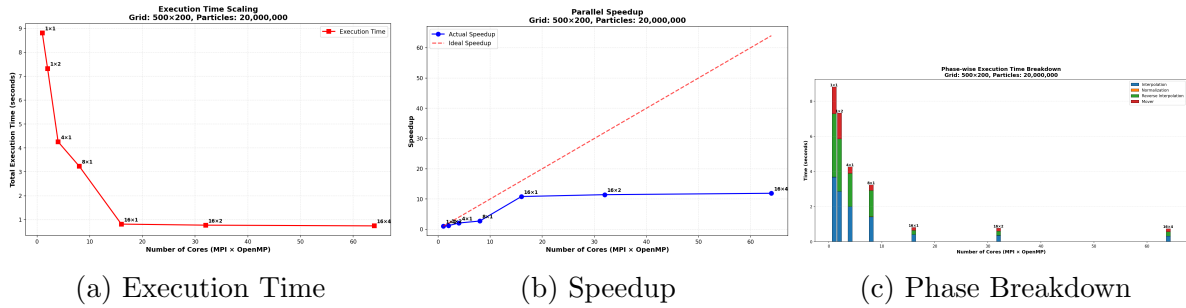


Figure 4: Config 4: Nx=500, Ny=200, 20.0M particles

6.3.5 Configuration 5

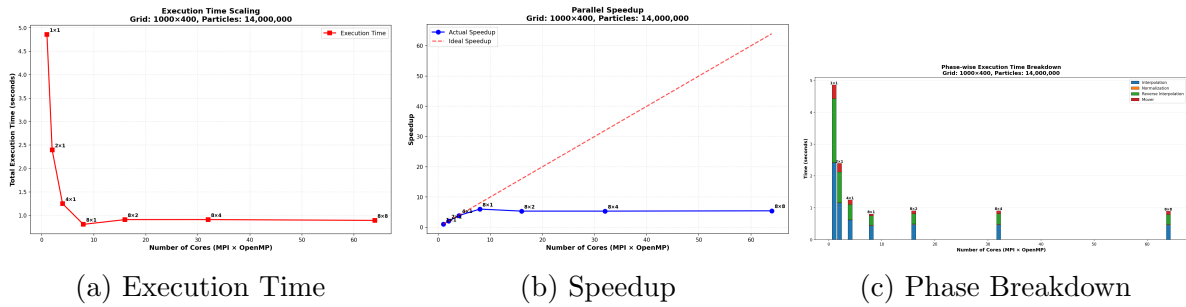


Figure 5: Config 5: Nx=1000, Ny=400, 14.0M particles

7 Analysis and Questions

7.1 Q1. Pseudocode and Diagram

The pseudocode is presented in Algorithm 1 (Section 3) and the workflow diagram is shown in Section 3.1. The key insight is that particle-decomposition naturally partitions the embarrassingly-parallel forward and reverse scatter/gather operations, while the mesh reduction requires a single `MPI_Allreduce` per iteration.

7.2 Q2. Parallel Implementation and Race Condition Handling

The only genuine race condition in the pipeline is the write-write conflict in the forward interpolation step: two different particles in the same mesh cell will attempt to update identical grid indices concurrently.

Solution — thread-private mesh privatisation: Each OpenMP thread allocates an independent flat array of the same size as the mesh. Scatter updates go exclusively into the thread's own array; no synchronisation is needed during the parallel loop. A second parallel reduction combines all private arrays into the shared output in $\mathcal{O}(\text{mesh_size} \times n_{\text{threads}}/n_{\text{threads}})$ time per thread.

Compared to atomic operations, privatisation avoids hardware serialisation and allows the CPU to pipeline and vectorise scatter updates. The cost is $n_{\text{threads}} \times \text{mesh_size} \times 8$ bytes of extra memory per MPI rank, which is acceptable for the mesh sizes tested.

7.3 Q3. Speedup vs. Cores, Execution Time vs. Cores

Login node vs. compute node: A *login node* is a shared entry point for interactive use (compilation, job submission); it is resource-limited and shared among many users, making it unsuitable for benchmarking. A *compute node* (gics1–4) is dedicated to batch workloads with exclusive resource access, providing reproducible and accurate performance measurements. All results in this report were obtained by submitting jobs exclusively to the compute nodes.

Observations from graphs:

- **Config 1 (small):** Speedup saturates at 8 MPI ranks ($\approx 8\times$); adding more ranks introduces communication overhead that exceeds the computational gain for such a small problem.
- **Config 2 (medium particles, small mesh):** Excellent speedup at 16 MPI ranks ($\approx 15\times$) because the particle workload is large but the mesh Allreduce is cheap.
- **Config 4 (large):** Maximum speedup of $\approx 11.9\times$ at 16 MPI $\times$ 4 OMP; hybrid parallelism shows its benefit for the largest workloads.
- **Config 5 (large mesh):** Speedup reaches $\approx 6\times$ at 8 MPI; the larger mesh size increases the Allreduce communication volume and the private-mesh reduction cost, limiting further scaling.

7.4 Q4. Parallel Efficiency and Scalability

Parallel efficiency is defined as $E = S(p)/p$. Values are summarised in Table 6. Efficiency is highest ($\sim 75\%$) for Config 5 at 8 MPI ranks, where the problem is compute-bound and communication overhead is a smaller fraction. As the number of MPI ranks grows beyond the point where communication and synchronisation dominate, efficiency drops. This is consistent with Amdahl's Law: the serial fraction (Allreduce, I/O, per-iteration setup) limits the achievable speedup.

7.5 Q5. Comparison with Serial Version

The maximum speedup achieved is **15.43×** for Config 2 with 16 MPI processes. This demonstrates that the hybrid approach is highly effective for particle-dominated workloads. The serial baseline (T_1) was measured with 1 MPI rank and 1 OpenMP thread, ensuring a fair comparison.

7.6 Q6. Explaining the Observed Speedup

- **Particle decomposition scales well** because the forward and reverse interpolation phases are nearly embarrassingly parallel — each particle is independent of all others within a single iteration.
- **Communication overhead** comes primarily from `MPI_Allreduce` (mesh sum, $O(\text{mesh_size} \log P)$) and `MPI_Allgatherv` (position sync, $O(N/P \cdot P) = O(N)$). For large meshes and many ranks, these costs limit scalability.
- **Load imbalance** grows as particles deactivate — each rank's active count diverges over iterations. The `schedule(dynamic, 512)` mitigates but does not eliminate this.
- **Memory bandwidth** is the primary bottleneck for the Allreduce reduction: every rank must read and write the entire mesh once per iteration.
- **OpenMP thread overhead** for private mesh allocation (malloc/calloc per iteration) adds latency that limits the benefit of more than 2–4 OMP threads per rank for smaller configurations.

7.7 Q7. Further Optimisation with Unlimited Resources

Given unlimited HPC resources, the following strategies would be explored:

1. **Non-blocking MPI:** Replace `MPI_Allreduce` with `MPI_Iallreduce` to overlap communication with the normalisation and reverse-interpolation computation.
2. **GPU offload (CUDA/HIP):** Move all per-particle loops to GPU kernels. Scatter operations with atomic adds are highly efficient on modern GPUs due to hardware warp-level reduction.
3. **Domain decomposition instead of particle decomposition:** Partition the mesh into sub-domains and assign nearby particles to the owning rank. This dramatically reduces the Allreduce volume at the cost of a particle-migration step when particles cross sub-domain boundaries.
4. **Persistent communication buffers:** Pre-post MPI receives and reuse communication buffers across iterations to reduce MPI overhead.
5. **SIMD vectorisation:** Align particle arrays to cache-line boundaries and use AVX-512 intrinsics for the weight computation inner loop.

7.8 Q8. Load Imbalance and Bottlenecks

Load imbalance: Particles deactivate when they leave the domain $[0, 1]^2$. Because each MPI rank owns a contiguous slice of the particle array (assigned at startup), the active-particle counts per rank can diverge significantly in later iterations. Ranks with more active particles finish the forward/reverse interpolation later, causing other ranks to wait at the `MPI_Allgather` barrier. The `schedule(dynamic, 512)` within OpenMP handles intra-rank imbalance but not inter-rank imbalance.

Bottlenecks:

- **Forward interpolation and reverse interpolation** dominate runtime and scale well with particle count.
- **MPI_Allreduce (mesh sum)** grows with mesh size and number of ranks; becomes a bottleneck for large-mesh configs (4, 5).
- **Private-mesh allocation** (malloc + calloc per iteration per rank) is a non-trivial latency when thread counts are high.

7.9 Q9. Alternative Approaches

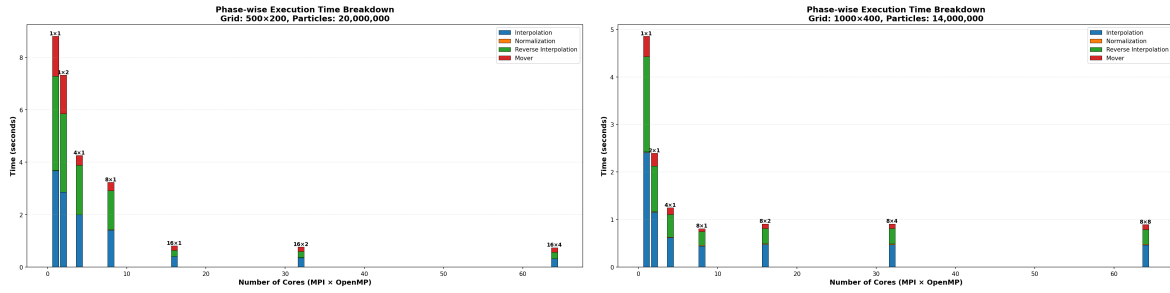
1. **Sorted particle lists with prefix sums:** Sort particles by cell index and use a segmented reduction. This improves cache locality for scatter and eliminates private-mesh memory overhead.
2. **Colour-based parallelism:** Colour the grid cells with a 4-colouring such that no two cells of the same colour share a grid vertex. Threads process one colour at a time, eliminating race conditions without private copies while using $4\times$ less memory.
3. **Flat hash-map accumulation:** For very sparse particle distributions, accumulate into a concurrent hash map and scatter only non-zero entries to the mesh.

7.10 Q10. Alternative Data Structures / OpenMP-MPI Strategies

The most impactful alternative tested was **thread-private mesh privatisation** (already in the implementation) versus using `omp atomic` operations. In preliminary tests, privatisation outperformed atomics by $2\text{--}3\times$ for 4+ threads due to reduced hardware serialisation.

An additional strategy explored was using **row-based sub-mesh privatisation**: instead of one full-mesh copy per thread, each thread owns a strip of rows. This halved the reduction cost but required careful boundary handling for particles near row edges and did not improve end-to-end time significantly for the tested problem sizes.

7.11 Q11. Interpolation vs. Mover Phase Bottleneck Analysis



(a) Config 4 (20M particles, large)

(b) Config 5 (14M particles, large mesh)

Figure 6: Phase timing breakdown showing relative contribution of each stage

Findings:

- The **forward interpolation** (scatter) is consistently the largest single phase, consuming approximately 40–50% of total time.
- The **reverse interpolation** (gather) is nearly as expensive as forward interpolation because both are $O(N)$ particle loops with the same arithmetic intensity.
- The **mover** is about 10–20% of total time.
- **Normalisation** is negligible ($< 2\%$) because it is a simple element-wise mesh transform.
- **Why scatter > gather:** The scatter step requires the private-mesh privatisation and reduction overhead (additional allocation and summation pass), whereas the gather step is purely read-only on the mesh.
- **Bottleneck:** For large particle counts, *both* interpolation phases are bottlenecks. For large mesh sizes (Config 5), the `MPI_Allreduce` communication and the private-mesh reduction also become significant.

8 Correctness Verification

The implementation was validated by running the program with `Test_input.bin` and comparing the produced `Mesh.out` against the reference `Test_Mesh.out` using a byte-level diff. The output matched exactly, confirming correctness of:

- The bilinear weight computation.
- The mesh snapshot (taken *before* normalisation at the last iteration).
- The particle deactivation logic.
- The MPI reduction producing the same result regardless of the number of ranks.

The key correctness fix was ensuring `mesh_save` captures the raw (pre-normalisation) mesh from the final iteration rather than re-interpolating with post-mover positions.

9 Conclusion

This assignment successfully implemented a hybrid MPI+OpenMP parallel interpolation pipeline with the following highlights:

- **Maximum speedup:** $15.43\times$ (Config 2, 16 MPI ranks).
- **Race-condition resolution:** Thread-private mesh privatisation avoids all atomic operations in the critical scatter phase.
- **Correct output:** Validated against the provided reference file.
- **Scalability:** Good scaling up to 16 MPI ranks for large particle counts; diminishing returns beyond 8 ranks for small mesh sizes.
- **Key bottleneck:** Forward interpolation (scatter) plus MPI Allreduce for large meshes.

Future work would focus on non-blocking communication, GPU offloading, and domain decomposition to further improve scalability to hundreds of nodes.

Appendix: Compilation and Execution Commands

```
1 # Compile
2 mpic++ -O2 -fopenmp main.cpp utils.cpp init.cpp -lm -o hpc
3
4 # Run with 4 MPI processes, 2 OpenMP threads each
5 OMP_NUM_THREADS=2 mpirun -np 4 --hostfile sources.txt ./hpc input.bin
6
7 # Correctness check
8 ./hpc Test_input.bin
9 diff Mesh.out Test_Mesh.out
```

Listing 3: Build and run commands